
User Manual

for MPC560XB GPT Driver

Document Number: UM13GPTASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Overview.....	9
2.2	About this Manual.....	9
2.3	Acronyms and Definitions.....	9
2.4	Reference List.....	10
2.5	Supported Derivatives.....	10
Chapter 3		
Driver		
3.1	Requirements.....	13
3.2	Driver Design Sumary.....	13
3.3	Additional Requirement and Deviations.....	14
3.4	Function Definitions.....	15
3.4.1	Function Gpt_Init.....	15
3.4.2	Function Gpt_DeInit.....	16
3.4.3	Function Gpt_EnableNotification.....	17
3.4.4	Function Gpt_DisableNotification.....	17
3.4.5	Function Gpt_DisableWakeup.....	18
3.4.6	Function Gpt_EnableWakeup.....	18
3.4.7	Function Gpt_GetTimeElapsed.....	19
3.4.8	Function Gpt_GetTimeRemaining.....	19
3.4.9	Function Gpt_SetMode.....	20
3.4.10	Function Gpt_StartTimer.....	21
3.4.11	Function Gpt_StopTimer.....	21
3.4.12	Function Gpt_GetVersionInfo.....	22
3.4.13	Function Gpt_CheckWakeup.....	22

Section number	Title	Page
3.4.14	Function Gpt_SetClockMode.....	23
3.5	Define Definitions.....	23
3.5.1	Define GPT_CER_UNINIT_INTERRUPT.....	23
3.5.2	Define GPT_CER_WRONGMODULE.....	23
3.5.3	Define GPT_PIT_INTERRUPT_ID.....	24
3.5.4	Define GPT_STM_INTERRUPT_ID.....	24
3.5.5	Define GPT_E_BUSY.....	24
3.5.6	Define GPT_E_ALREADY_INITIALIZED.....	24
3.5.7	Define GPT_E_UNINIT.....	24
3.5.8	Define GPT_E_PARAM_CHANNEL.....	24
3.5.9	Define GPT_E_PARAM_MODE.....	24
3.5.10	Define GPT_E_PARAM_VALUE.....	25
3.5.11	Define GPT_E_PARAM_CLOCK_MODE.....	25
3.5.12	Define GPT_CH_MODE_CONTINUOUS.....	25
3.5.13	Define GPT_CH_MODE_ONESHOT.....	25
3.6	Structure Definitions.....	25
3.6.1	Structure Gpt_ConfigType.....	25
3.6.2	Structure Gpt_LLD_ChannelConfigType.....	26
3.6.3	Structure Std_VersionInfoType.....	27
3.6.4	Structure Gpt_LLD_ChannelInfoType.....	27
3.7	Enum Definitions.....	28
3.7.1	Enumeration Gpt_ModeType.....	28
3.7.2	Enumeration Gpt_ChannelStatusType.....	28
3.7.3	Enumeration Gpt_ClockModeType.....	28
3.8	Typedef Definitions.....	29
3.8.1	Typedef Gpt_ChannelModeType.....	29
3.8.2	Typedef Gpt_ChannelType.....	29
3.8.3	Typedef Gpt_NotifyType.....	29
3.8.4	Typedef Gpt_PrescaleType.....	29

Section number	Title	Page
3.8.5	Typedef Gpt_ValueType.....	30
3.8.6	Typedef Gpt_WakeUpType.....	30
3.9	Symbolic Names DISCLAIMER.....	30
3.10	Configuration Parameters.....	30
3.10.1	Pre-Compile parameters.....	31
3.10.1.1	GptDevErrorDetect.....	32
3.10.1.2	GptReportWakeupSource.....	32
3.10.1.3	GptDeinitApi.....	33
3.10.1.4	GptEnableDisableNotificationApi.....	33
3.10.1.5	GptTimeElapsedApi.....	33
3.10.1.6	GptTimeRemainingApi.....	34
3.10.1.7	GptVersionInfoApi.....	34
3.10.1.8	GptWakeupFunctionalityApi.....	34
3.10.1.9	GptEnableDualClockMode.....	34
3.10.1.10	GptClockReferencePoint.....	35
3.10.2	Post-Build Parameters.....	35
3.10.2.1	Structure Gpt_ConfigType.....	36
3.10.2.1.1	Structure Gpt_LLD_ChannelConfigType.....	36
3.10.2.2	GptChannelId.....	37
3.10.2.3	GptSTMChannelPrescale.....	37
3.10.2.4	GptHwChannel.....	38
3.10.2.5	GptChannelMode.....	39
3.10.2.6	GptChannelTickFrequency.....	40
3.10.2.7	GptChannelPrescale.....	40
3.10.2.8	GptFreezeEnable.....	41
3.10.2.9	GptEnableWakeup.....	41
3.10.2.10	GptNotification.....	42
3.10.2.11	Gpt_ChannelPrescale_Alternate.....	43
3.10.2.12	GptChannelClkSrcRef.....	43

Section number	Title	Page
3.10.2.13	GptWakeupSourceRef.....	43
3.10.2.14	GptDisableEcumWakeupSourceNotification.....	44

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	17.02.2012	Surjit Kumar	Update for Beta 0.9.0
1.1	14.05.2013	Subramanya MN	Update for RTM 1.0.0
1.2	06.06.2014	Nguyen Viet Hung	Update for RTM 1.0.1



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR General Purpose Timer (GPT) driver for MPC560XB. AUTOSAR GPT driver configuration parameters and deviations from the specification are described in GPT Driver chapter. AUTOSAR GPT driver requirements and APIs are described in the AUTOSAR GPT driver software specification document. (Table 1 2)

2.1 Overview

AUTOSAR (Automotive Open System Architecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

2.2 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

2.3 Acronyms and Definitions

Table 2-1. Acronyms and Definitions

Term	Definition
API	Application Programming Interface

Table continues on the next page...

Table 2-1. Acronyms and Definitions (continued)

Term	Definition
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
GPT	General Purpose Timer
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit
STM	System Timer Module
PIT	Periodic Interrupt Timer
RTC	Real Time Clock
API	Autonomous Periodic Interrupt

2.4 Reference List

Table 2-2. Reference List

Title	Version
AUTOSAR GPT Driver Software Specification Document	R3.2.0 Rev0003
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013
Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

2.5 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor :

Table 2-3. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as MPC560XB.

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003GPT Driver Software Specification document (See Table **Reference List**).

3.2 Driver Design Summary

The GPT Driver implements maximum 77 channels on 5 MPC560XB peripherals. maximum 64 channels are implemented on 2 the Enhanced Modular IO Subsystem (eMIOS) module.

4 channels are implemented on System Timer Module (STM).

8 channels are implemented on Periodic Interrupt Timer (PIT).

1 channel is implemented on the Real Time Clock (RTC_API) module.

eMIOS channels

- 64 x 16-bit counters per unit

PIT timer channel

- Independent timeout periods for each timer

RTC_API channel

- a 32 bits counter with 3 optional clock sources

STM timer channel

- One 32-bit up counter with 8-bit prescaler (1 to 256).
- Four 32-bit compare channels
- Independent interrupt source for each channel

NOTE

The table provides information regarding the Timer channels available for the various derivatives across different packages in MPC560XB family. This table lists only the supported packages by GPT driver.

Table 3-1. GPT Hardware channels availability for MPC560XB family

Device	Total eMIOS channels	Total PIT channels	Total STM channels	Total RTC_API channels
>mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxll_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxll_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxll_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxll_lqfp144 mpc5605bxll_lqfp176 mpc5606bxll_lqfp100 mpc5606bxll_lqfp144 mpc5606bxll_lqfp176 mpc5607bxll_lqfp176 mpc5607bxmg_mapbga208	17 ch, 16-bit	8 ch, 32-bit	4 ch, 32-bit	1 ch, 32-bit

3.3 Additional Requirement and Deviations

The driver deviates from the AUTOSAR Driver software specification in some places.

Deviation from the AUTOSAR GPT Driver software specification are listed in table **Gpt Deviation table**

Table 3-2. Deviations Status Column Description

Term	Definition
N/A	Not Available

Table continues on the next page...

Table 3-2. Deviations Status Column Description (continued)

Term	Definition
N/V	Not Verifiable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented
N/R	Unclear requirement

Table 3-3. Gpt Deviations Table

Requirement	Status	Description	Notes
GPT185	N/R	If a timer channel is configured in "one-shot mode": If the timer has reached the target time (timer value = target time), the timer shall stop automatically and maintain its timer value unchanged. The channel state shall change from "running" to "expired".	Hardware requirement only for up counters.
GPT186	N/R	If a timer channel is configured in "continuous mode": If the timer has reached the target time (timer value = target time), the timer shall continue running with the value "0" at next timer tick. So, the time interval of the recurrence is: target time + 1. This interval shall be independently of implementation, e.g. interrupt delays.	Hardware requirement only for up counters.
GPT273	N/I	If source code for caller and callee of Gpt_GetVersionInfo is available, the GPT module should realize Gpt_GetVersionInfo as a macro, defined in the module's header file.	Unclear formulated.
GPT329	N/R	A timer channel starts counting at value zero.	Hardware requirement only for up counters.
GPT336	N/S	Variables available for debugging shall be described in the respective Basic Software Module Description.	Not a driver requirement.

3.4 Function Definitions

APIs of all functions supported by the driver are as per AUTOSAR GPT Driver software specification Version 4.0 Rev0003

3.4.1 Function Gpt_Init

This function returns the version information of this module.

Prototype: `void Gpt_Init(const Gpt_ConfigType *configPtr);`

Table 3-4. Gpt_Init Arguments

Type	Name	Direction	Description
Commented parameter versioninfo does not exist in function Gpt_Init.	versioninfo	output	- pointer to location to store version info
const Gpt_ConfigType *	configPtr	input	Pointer to a selected configuration structure

This service returns the version information of this module. The version information includes: Module Id Vendor Id Vendor specific version numbers (BSW00407).

Note

Violates GPT_H_REF_2 Repeated include file
MemMap.h Violates GPT_C_REF_1 Only preprocessor statements
and comments before "#include" Violates GPT_C_REF_2
Repeated include file MemMap.h

Service for driver initialization. The Initialization function shall initialize all relevant registers of the configured hardware with the values of the structure referenced by the parameter ConfigPtr. All time units used within the API services of the GPT driver shall be of the unit ticks. This function shall only initialize the configured resources. Resources that are not configured in the configuration file shall not be touched. The following rules regarding initialization of controller registers shall apply to the GPT Driver implementation: [1] If the hardware allows for only one usage of the register, the driver module implementing that functionality is responsible for initializing the register [2] If the register can affect several hardware modules and if it is an I/O register it shall be initialized by the PORT driver [3] If the register can affect several hardware modules and if it is not an I/O register it shall be initialized by the MCU driver [4] One-time writable registers that require initialization directly after reset shall be initialized by the startup code [5] All other registers shall be initialized by the startup code

pre:: On/Off by the configuration parameter: GPT_VERSION_INFO_API

3.4.2 Function Gpt_DeInit

GPT driver de-initialization function.

Prototype: void Gpt_DeInit(void);

Service for deinitializing all hardware timer channels to their power on reset state. The state of the peripheral after DeInit shall be the same as after power on reset. The service influences only the peripherals, which are allocated by static configuration and the

runtime configuration set passed by the previous call of `Gpt_Init()`. The driver needs to be initialized before calling `Gpt_DeInit()`. Otherwise, the function `Gpt_DeInit` shall raise the development error `GPT_E_UNINIT` and leave the desired deinitialization functionality without any action.

pre:: On/Off by the configuration parameter: `GPT_DEINIT_API`

3.4.3 Function `Gpt_EnableNotification`

GPT driver function for enabling the notification for a timer channel.

Prototype: `void Gpt_EnableNotification(Gpt_ChannelType channel);`

Table 3-5. `Gpt_EnableNotification` Arguments

Type	Name	Direction	Description
<code>Gpt_ChannelType</code>	channel	input	channel id

Service for enabling the notification for a channel during runtime. This function can be called, while the timer is already running. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. The driver needs to be initialized before calling `Gpt_EnableNotification()`. Otherwise, the function `Gpt_EnableNotification` shall raise the development error `GPT_E_UNINIT`.

pre:: On/Off by the configuration parameter:
`GPT_ENABLE_DISABLE_NOTIFICATION_API`

3.4.4 Function `Gpt_DisableNotification`

GPT driver function for disabling the notification for a timer channel.

Prototype: `void Gpt_DisableNotification(Gpt_ChannelType channel);`

Table 3-6. `Gpt_DisableNotification` Arguments

Type	Name	Direction	Description
<code>Gpt_ChannelType</code>	channel	input	channel id

Service for disabling the notification for a channel during runtime. This function can be called, while the timer is already running. When disabled, no notification will be sent. When re-enabled again, the user will not be notified of events, occurred while notifications have been disabled. Usage of re-entrant capability is only allowed if the

callers take care that there is no simultaneous usage of the same channel. The driver needs to be initialized before calling `Gpt_DisableNotification()`. Otherwise, the function shall raise the development error `GPT_E_UNINIT`.

pre:: On/Off by the configuration parameter:
`GPT_ENABLE_DISABLE_NOTIFICATION_API`

3.4.5 Function Gpt_DisableWakeup

GPT driver function for disabling the wakeup interrupt invocation for a timer channel.

Prototype: `void Gpt_DisableWakeup(Gpt_ChannelType channel);`

Table 3-7. Gpt_DisableWakeup Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	channel id

This service shall disable the wakeup interrupt invocation of a single GPT channel. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. The driver needs to be initialized before calling `Gpt_DisableWakeup()`. Otherwise, the function `Gpt_DisableWakeup` shall raise the development error `GPT_E_UNINIT`.

pre:: On/Off by the configuration parameter: `GPT_WAKEUP_FUNCTIONALITY_API`

3.4.6 Function Gpt_EnableWakeup

GPT driver function for enabling the wakeup interrupt invocation for a timer channel.

Prototype: `void Gpt_EnableWakeup(Gpt_ChannelType channel);`

Table 3-8. Gpt_EnableWakeup Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	channel id

This service shall enable the wakeup interrupt invocation of a single GPT channel. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. The driver needs to be initialized before calling `Gpt_EnableWakeup()`. Otherwise, the function `Gpt_EnableWakeup` shall raise the development error `GPT_E_UNINIT`.

pre:: On/Off by the configuration parameter: GPT_WAKEUP_FUNCTIONALITY_API

3.4.7 Function Gpt_GetTimeElapsed

GPT driver function for fetching the elapsed timer value.

Prototype: `Gpt_ValueType Gpt_GetTimeElapsed(Gpt_ChannelType channel);`

Table 3-9. Gpt_GetTimeElapsed Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	- channel id

Return: Gpt_ValueType - Elapsed Time in number of ticks

Service for querying the time already elapsed. In one shot mode, this is the value relative to the point in time, the channel has been started with Gpt_StartTimer (calculated by the normal operation function by subtracting the current minus the initial timer value and returning the absolute value). In continuous mode, the function returns the timer value relative to the last timeout/the start of the channel. All time units used within the API services of the GPT driver shall be of the unit ticks. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. To get times out of register values it is necessary to know the oscillator frequency, pre prescalers and so on. Since these settings are made in MCU and/or in other modules it is not possible to calculate such times. Hence the conversions between time and ticks shall be part of an upper layer. The driver needs to be initialized before calling `Gpt_GetTimeElapsed()`. Otherwise, the function shall raise the development error GPT_E_UNINIT and return 0.

pre:: On/Off by the configuration parameter: GPT_TIME_ELAPSED_API

Note

Violates GPT_C_REF_3, taking address of near auto variable

3.4.8 Function Gpt_GetTimeRemaining

GPT driver function for fetching the remaining timer value.

Prototype: `Gpt_ValueType Gpt_GetTimeRemaining(Gpt_ChannelType channel);`

Table 3-10. Gpt_GetTimeRemaining Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	- channel id

Return: Gpt_ValueType - Remaining Time in number of ticks

This function returns the timer value remaining until the next timeout period will expire (calculated by the normal operation function by subtracting the timeout minus the current timer value and returning the absolute value) All time units used within the API services of the GPT driver shall be of the unit ticks. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. To get times out of register values it is necessary to know the oscillator frequency, prescalers and so on. Since these settings are made in MCU and/or in other modules it is not possible to calculate such times. Hence the conversions between time and ticks shall be part of an upper layer. The driver needs to be initialized before calling `Gpt_GetTimeRemaining()`. Otherwise, the function shall raise the development error `GPT_E_UNINIT` and return 0.

pre:: On/Off by the configuration parameter: `GPT_TIME_REMAINING_API`

Note

Violates `GPT_C_REF_3`, taking address of near auto variable

3.4.9 Function Gpt_SetMode

GPT driver function for setting the operation mode.

Prototype: `void Gpt_SetMode(Gpt_ModeType mode);`

Table 3-11. Gpt_SetMode Arguments

Type	Name	Direction	Description
Gpt_ModeType	mode	input	operation mode

Service for GPT mode selection. This service shall set the operation mode to the given mode parameter . When sleep mode is requested, the ECU State Manager calls `Gpt_SetMode` with mode parameter "GPT_MODE_SLEEP" and prepares the GPT for sleep mode. The MCU Driver is then putting the controller into SLEEP mode The driver needs to be initialized before calling `Gpt_SetMode()`. Otherwise, the function `Gpt_SetMode` shall raise the development error `GPT_E_UNINIT`.

pre:: On/Off by the configuration parameter: `GPT_WAKEUP_FUNCTIONALITY_API`

3.4.10 Function Gpt_StartTimer

GPT driver function for starting a timer channel.

Prototype: `void Gpt_StartTimer(Gpt_ChannelType channel, Gpt_ValueType value);`

Table 3-12. Gpt_StartTimer Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	channel id
Gpt_ValueType	value	input	timeout period (in number of ticks) after a notification shall occur

The function Gpt_StartTimer shall start the selected timer channel with a defined timeout period. The function Gpt_StartTimer shall invoke the configured notification for that channel (see also GPT292) after the timeout period referenced via the parameter value (if enabled). All time units used within the API services of the GPT driver shall be of the unit ticks. In production mode no error is generated. The rational is that it adds no additional functionality to the driver. In this case the timer will be restarted with the timeout value, given as a parameter to the service. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. To get times out of register values it is necessary to know the oscillator frequency, prescalers and so on. Since these settings are made in MCU and/or in other modules it is not possible to calculate such times. Hence the conversions between time and ticks shall be part of an upper layer. The driver needs to be initialized before calling Gpt_StartTimer(). Otherwise, the function Gpt_StartTimer shall raise the development error GPT_E_UNINIT.

3.4.11 Function Gpt_StopTimer

GPT driver function for stopping a timer channel.

Prototype: `void Gpt_StopTimer(Gpt_ChannelType channel);`

Table 3-13. Gpt_StopTimer Arguments

Type	Name	Direction	Description
Gpt_ChannelType	channel	input	channel id

Service for stopping the selected timer channel Stopping a timer channel, not been started before will not return a development error Timer channels configured in one shot mode are stopped automatically, when the timeout period has expired. Usage of re-entrant capability is only allowed if the callers take care that there is no simultaneous usage of the same channel. The driver needs to be initialized before calling `Gpt_StopTimer()`. Otherwise, the function shall raise the development error `GPT_E_UNINIT`.

3.4.12 Function Gpt_GetVersionInfo

This function returns the version information of this module.

Prototype: `void Gpt_GetVersionInfo(Std_VersionInfoType *versioninfo);`

Table 3-14. Gpt_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	output	- pointer to location to store version info

This service returns the version information of this module. The version information includes: Module Id Vendor Id Vendor specific version numbers (BSW00407).

Note

Violates GPT_H_REF_2 Repeated include file
MemMap.h Violates GPT_C_REF_1 Only preprocessor statements
and comments before "#include" Violates GPT_C_REF_2
Repeated include file MemMap.h

pre:: On/Off by the configuration parameter: `GPT_VERSION_INFO_API`

3.4.13 Function Gpt_CheckWakeup

GPT driver function for checking if a wakeup capable GPT channel is the source for a wakeup event.

Prototype: `void Gpt_CheckWakeup(EcuM_WakeupSourceType wakeupSource);`

Table 3-15. Gpt_CheckWakeup Arguments

Type	Name	Direction	Description
EcuM_WakeupSourceType	wakeupSource	input	wakeup source

Checks if a wakeup capable GPT channel is the source for a wakeup event and calls the ECU state manager service EcuM_SetWakeupEvent in case of a valid GPT channel wakeup event. The driver needs to be initialized before calling Gpt_Cbk_CheckWakeup(). Otherwise, the function Gpt_Cbk_CheckWakeup shall raise the development error GPT_E_UNINIT.

pre:: On/Off by the configuration parameter: GPT_WAKEUP_FUNCTIONALITY_API

3.4.14 Function Gpt_SetClockMode

This function changes the channel prescaler.

Prototype: void Gpt_SetClockMode(Gpt_ClockModeType clkMode);

Table 3-16. Gpt_SetClockMode Arguments

Type	Name	Direction	Description
Gpt_ClockModeType	clkMode	input	prescaler setting (NORMAL or ALTERNATE)

This function sets all channels prescalers based on the input mode.

pre:: Gpt_Init must be called before.

Note

Implements NonAutosar functionality

3.5 Define Definitions

3.5.1 Define GPT_CER_UNINIT_INTERRUPT

error in interrupt function

Definition: #define GPT_CER_UNINIT_INTERRUPT (uint8)0x02U

3.5.2 Define GPT_CER_WRONGMODULE

wrong module index in switch statement

Definition: #define GPT_CER_WRONGMODULE (uint8)0x01U

3.5.3 Define GPT_PIT_INTERRUPT_ID

API service ID for Gpt_Pit_ProcessInterrupt function.

Definition:`#define GPT_PIT_INTERRUPT_ID (uint8)0x02U`

3.5.4 Define GPT_STM_INTERRUPT_ID

API service ID for Gpt_Stm_ProcessInterrupt function.

Definition:`#define GPT_STM_INTERRUPT_ID (uint8)0x03U`

3.5.5 Define GPT_E_BUSY

function called when timer channel is still running

Definition:`#define GPT_E_BUSY (uint8)0x0BU`

3.5.6 Define GPT_E_ALREADY_INITIALIZED

initialization called when already initialized

Definition:`#define GPT_E_ALREADY_INITIALIZED (uint8)0x0DU`

3.5.7 Define GPT_E_UNINIT

function called without module initialization

Definition:`#define GPT_E_UNINIT (uint8)0x0AU`

3.5.8 Define GPT_E_PARAM_CHANNEL

function called for invalid channel

Definition:`#define GPT_E_PARAM_CHANNEL (uint8)0x14U`

3.5.9 Define GPT_E_PARAM_MODE

function called with invalid mode param.

Definition: `#define GPT_E_PARAM_MODE (uint8)0x1FU`

3.5.10 Define GPT_E_PARAM_VALUE

function called with parameter value out of range

Definition: `#define GPT_E_PARAM_VALUE (uint8)0x15U`

3.5.11 Define GPT_E_PARAM_CLOCK_MODE

API Gpt_SetClockMode service called with wrong parameter.

Definition: `#define GPT_E_PARAM_CLOCK_MODE 0x7AU`

3.5.12 Define GPT_CH_MODE_CONTINUOUS

continous mode.

Definition: `#define GPT_CH_MODE_CONTINUOUS ((uint8)0x01U)`

3.5.13 Define GPT_CH_MODE_ONESHOT

one shot mode.

Definition: `#define GPT_CH_MODE_ONESHOT ((uint8)0x00U)`

3.6 Structure Definitions

Data structures supported by the driver are as per AUTOSAR GPT Driver software specification Version 4.0 Rev0003.

3.6.1 Structure Gpt_ConfigType

Gpt configuration type - this is the type of the data structure including the configuration set required for initializing the GPT driver.

Declaration

```
typedef struct
{
    uint8 Gpt_Channel_Count,
    const Gpt_LLD_ChannelConfigType * Gpt_ChannelConfigPtr
} Gpt_ConfigType;
```

Table 3-17. Structure Gpt_ConfigType member description

Member	Description
Gpt_Channel_Count	the number of GPT channels (configured in tresos plugin builder)
Gpt_ChannelConfigPtr	brief pointer to the GPT channel configuration

3.6.2 Structure Gpt_LLD_ChannelConfigType

Gpt channel configuration type.

Declaration

```
typedef struct
{
    Gpt_ChannelType Gpt_HW_Channel,
    Gpt_ChannelType Gpt_HW_Module,
    Gpt_ChannelModeType Gpt_ChannelMode,
    uint8 Gpt_ChannelClkSrc,
    Gpt_PrescaleType Gpt_ChannelPrescale,
#if (GPT_DUAL_CLOCK_MODE == STD_ON)
    Gpt_PrescaleType Gpt_ChannelPrescale_Alternate,
#endif
    boolean Gpt_FreezeEnable,
    boolean Gpt_EnableWakeup,
    Gpt_NotifyType Gpt_Notification,
    Gpt_WakeUpType Gpt_ChannelWakeUpInfo
} Gpt_LLD_ChannelConfigType;
```

Table 3-18. Structure Gpt_LLD_ChannelConfigType member description

Member	Description
Gpt_HW_Channel	GPT hw channel ID.
Gpt_HW_Module	GPT hw module used.
Gpt_ChannelMode	GPT channel mode.
Gpt_ChannelClkSrc	GPT clock source.
Gpt_ChannelPrescale	GPT ch prescaler value.
Gpt_ChannelPrescale_Alternate	ch 2nd prescaler val
Gpt_FreezeEnable	GPT ch freeze enable.

Table continues on the next page...

Table 3-18. Structure Gpt_LLD_ChannelConfigType member description (continued)

Member	Description
Gpt_EnableWakeup	GPT ch WakeUp enable.
Gpt_Notification	pointer to ISR function
Gpt_ChannelWakeUpInfo	EcuM wake up reference.

Note: Not all the parameters in this structure are available for all the possible hardware IPs

3.6.3 Structure Std_VersionInfoType

This type shall be used to request the version of a BSW module using the "ModuleName"_GetVersionInfo() function.

Declaration

```
typedef struct
{
    uint16 moduleID,
    uint8 sw_major_version,
    uint8 sw_minor_version,
    uint8 sw_patch_version,
    uint16 vendorID
} Std_VersionInfoType;
```

Table 3-19. Structure Std_VersionInfoType member description

Member	Description
moduleID	BSW module ID.
sw_major_version	BSW module software major version.
sw_minor_version	BSW module software minor version.
sw_patch_version	BSW module software patch version.
vendorID	vendor ID

3.6.4 Structure Gpt_LLD_ChannelInfoType

Low level configuration for channel specific parameters.

Declaration

```
typedef struct
{
    Gpt_ValueType Gpt_ReloadValue,
    Gpt_ValueType Gpt_StopValue,
    uint8 BitInfo
} Gpt_LLD_ChannelInfoType;
```

Table 3-20. Structure Gpt_LLD_ChannelInfoType member description

Member	Description
Gpt_ReloadValue	It represents the timeout value of the channel
Gpt_StopValue	It represents, in case of a stopped channel, the stop value
BitInfo	stores information about channel mode, wake up, notification - each on 1 bit

3.7 Enum Definitions

Data Enums supported by the driver are as per AUTOSAR GPT Driver software specification Version 4.0 Rev0003.

3.7.1 Enumeration Gpt_ModeType

This enumerated type allows the selection of different power modes.

Table 3-21. Enumeration Gpt_ModeType Values

Value	Description
GPT_MODE_NORMAL = 0	GPT Normal operation mode of the GPT.
GPT_MODE_SLEEP	GPT sleep mode.

3.7.2 Enumeration Gpt_ChannelStatusType

This enumerated type allows the selection of channel status type.

Table 3-22. Enumeration Gpt_ChannelStatusType Values

Value	Description
GPT_STATUS_UNINITIALIZED = 0	GPT channel status - uninitialized.
GPT_STATUS_INITIALIZED	GPT channel status - initialized.
GPT_STATUS_STOPPED	GPT channel status - stopped.
GPT_STATUS_EXPIRED	GPT channel status - expired.
GPT_STATUS_RUNNING	GPT channel status - running.

3.7.3 Enumeration Gpt_ClockModeType

Prescaler type.

Table 3-23. Enumeration Gpt_ClockModeType Values

Value	Description
GPT_NORMAL = 0x0U	
GPT_ALTERNATE = 0x1U	

3.8 Typedef Definitions

Data Typedefs supported by the driver are as per AUTOSAR GPT Driver software specification Version 4.0 Rev0003.

3.8.1 Typedef Gpt_ChannelModeType

Gpt channel mode type. Indicates of whether the channel mode is "CONTINUOUS" or "ONE SHOT".

Type: uint32

3.8.2 Typedef Gpt_ChannelType

Gpt channel ID data type.

Type: uint8

3.8.3 Typedef Gpt_NotifyType

Gpt channel notification type. The callback notifications shall be configurable as pointers to user defined functions within the configuration structure.

Type: void(*)

3.8.4 Typedef Gpt_PrescaleType

Gpt prescale type. It specifies the GPT module specific prescaler factor per channel.

Type: uint32

3.8.5 Typedef Gpt_ValueType

Gpt timeout value data type. Used for reading the current timer value/setting periodic timer values (in number of ticks) up to hours.

Type: uint32

3.8.6 Typedef Gpt_WakeUpType

Gpt Wake up type.

Type: uint32

3.9 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing #ifdefs arguments).

3.10 Configuration Parameters

As per the AUTOSAR specification the driver has two types of configurations parameters: **Pre-Compile** parameters and **Post-Build** parameters. Pre-Compile parameters are stored in the file Gpt_Cfg.h and Gpt_Cfg.c. Post-Build parameters are stored in the file Gpt_PBcfg.c.

The files to be used for different configuration types are listed below:

1. **Variant PC:** Gpt_Cfg.h, Gpt_Cfg.c
2. **Variant PB:** Gpt_Cfg.h, Gpt_PBcfg.c
3. **Variant LT:** Not Applicable

A section for **Gpt_PBcfg.c** file is needed in linker file to place the post build configuration in desired location. Please refer to section-8 of AUTOSAR_SWS_C_ImplementationRules file for complete details on configuration types.

3.10.1 Pre-Compile parameters

The following picture gives an example of configuration screen for all GPT precompile parameters in Tresos® Studio configuration tool from Tresos Elektrobit Tresos Release 14.2.1 (b140128-1223)



Figure 3-1. PreCompile_GptDriverConfiguration

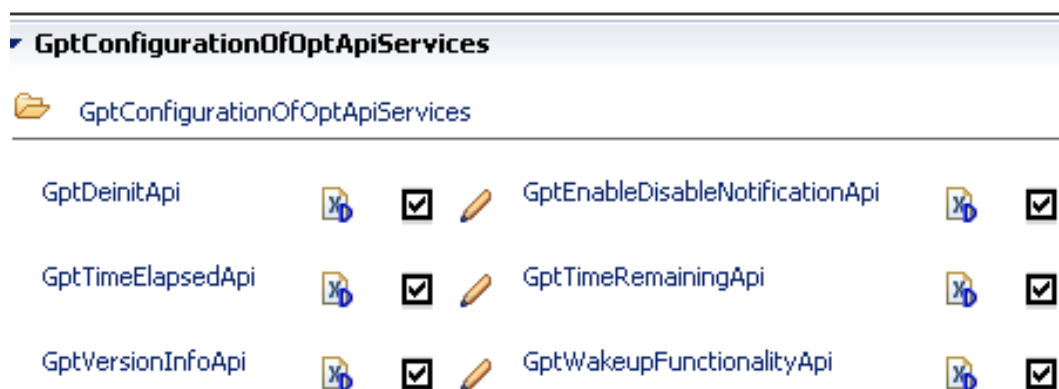
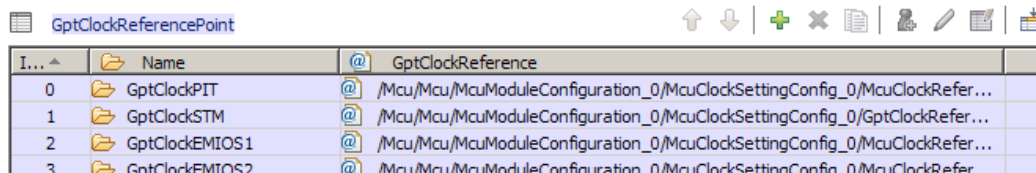


Figure 3-2. PreCompile_GptConfigurationOfOptApiServices



Figure 3-3. GptNonAutosar



I...	Name	@	GptClockReference
0	GptClockPIT	@	/Mcu/Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/McuClockRefer...
1	GptClockSTM	@	/Mcu/Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/GptClockRefer...
2	GptClockEMIOS1	@	/Mcu/Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/McuClockRefer...
3	GptClockEMIOS2	@	/Mcu/Mcu/McuModuleConfiguration_0/McuClockSettingConfig_0/McuClockRefer...

Figure 3-4. GptClockReferencePoint

NOTE

If pre-compile variant is preferred,

- IMPLEMENTATION_CONFIG_VARIANT in Tresos GUI should be selected as “VariantPreCompile” as shown below.



Figure 3-5. Config-variant

- Gpt_Cfg.c should be compiled
- Gpt_PBcfg.c should not be compiled

3.10.1.1 GptDevErrorDetect

Table 3-24. GptDevErrorDetect

Description	Enables/Disables development error detection. Defines whether support of development error detection should be included at compile time (STD_ON) or excluded (STD_OFF). When enabled API parameter checking is done at runtime.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	#define GPT_DEV_ERROR_DETECT STD_OFF / STD_ON
NOTE	Setting this control to false will generate code that is reduced, but some Autosar requirements are not tested (no Det errors are reported in this way).

3.10.1.2 GptReportWakeupSource

Table 3-25. GptReportWakeupSource

Description	Enables/Disables wakeup source reporting
Class	Autosar Parameter
Range	True, False
Default	True

Table continues on the next page...

Table 3-25. GptReportWakeupSource (continued)

Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_REPORT_WAKEUP_SOURCE STD_OFF / STD_ON</code>

3.10.1.3 GptDeinitApi

Table 3-26. GptDeinitApi

Description	Adds / removes the service Gpt_DeInit() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_DEINIT_API STD_OFF / STD_ON</code>

3.10.1.4 GptEnableDisableNotificationApi

Table 3-27. GptEnableDisableNotificationApi

Description	Adds / removes the services Gpt_EnableNotification() and Gpt_DisableNotification from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_ENABLE_DISABLE_NOTIFICATION_API STD_OFF / STD_ON</code>

3.10.1.5 GptTimeElapsedApi

Table 3-28. GptTimeElapsedApi

Description	Adds / removes the service Gpt_GetTimeElapsed() from the code
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_TIME_ELAPSED_API STD_OFF / STD_ON</code>

3.10.1.6 GptTimeRemainingApi

Table 3-29. GptTimeRemainingApi

Description	Adds / removes the service Gpt_GetTimeRemaining() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_TIME_REMAINING_API STD_OFF / STD_ON</code>

3.10.1.7 GptVersionInfoApi

Table 3-30. GptVersionInfoApi

Description	Adds / removes the service Gpt_GetVersionInfo() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_VERSION_INFO_API STD_OFF / STD_ON</code>

3.10.1.8 GptWakeupFunctionalityApi

Table 3-31. GptWakeupFunctionalityApi

Description	Adds / removes the services Gpt_SetMode(), Gpt_EnableWakeup() Gpt_DisableWakeup() and Gpt_Cbk_CheckWakeup() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_WAKEUP_FUNCTIONALITY_API STD_OFF / STD_ON</code>

3.10.1.9 GptEnableDualClockMode

Table 3-32. GptEnableDualClockMode

Description	Enables prescaler settings at mode transition.
--------------------	--

Table continues on the next page...

Table 3-32. GptEnableDualClockMode (continued)

Class	Implementation specific Non-Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<code>#define GPT_DUAL_CLOCK_MODE STD_OFF / STD_ON</code>

3.10.1.10 GptClockReferencePoint

Table 3-33. GptClockReferencePoint

Description	The parameter represents a reference to a container of the type McuClockReferencePoint (defined in module MCU).
Class	Autosar Parameter
Range	NA
Default	1.0
Source File	NA

3.10.2 Post-Build Parameters

Post Build parameters, their possible values and their meaning are described in the following text. The Post Build parameters are implemented as constant structures and arrays stored in flash memory of the MCU.

The following picture gives an example of configuration screen for all postbuild parameters in Tresos® Studio configuration tool from Tresos Elektrobit Tresos Release 14.2.1 (b140128-1223)

Configuration Parameters

GptChannelId (0 -> 4294967295)	1
GptHwChannel	STM_CH_2
GptChannelMode	GPT_CH_MODE_CONTINUOUS
GptChannelTickFrequency (0 -> ...)*	1.0
GptChannelTickValueMax	4294967296
EtimerChannelClkSrc (0 -> 22)	0
EtimerBusClkPrescale_Alternate (0 -> 128)	0
GptFreezeEnable	☒ GptEnableWakeup ☐
GptNotification	GptTestCallback02
GptChannelClkSrcRef*	/Gpt/Gpt/GptDriverConfiguration/STMClock
▼ GptWakeupConfiguration	
GptWakeupConfiguration	
GptWakeupSourceRef	

Figure 3-6. Post-Build-parameter

3.10.2.1 Structure Gpt_ConfigType

Gpt configuration type - this is the type of the data structure including the configuration set required for initializing the GPT driver.

Declaration

```
typedef struct
{
    uint8 Gpt_Channel_Count,
    const Gpt_LLD_ChannelConfigType * Gpt_ChannelConfigPtr
} Gpt_ConfigType;
```

Table 3-34. Structure Gpt_ConfigType member description

Member	Description
Gpt_Channel_Count	the number of GPT channels (configured in tresos plugin builder)
Gpt_ChannelConfigPtr	brief pointer to the GPT channel configuration

3.10.2.1.1 Structure Gpt_LLD_ChannelConfigType

Gpt channel configuration type.

Declaration

```
typedef struct
{
    Gpt_ChannelType Gpt_HW_Channel,
    Gpt_ChannelType Gpt_HW_Module,
    Gpt_ChannelModeType Gpt_ChannelMode,
    uint8 Gpt_ChannelClkSrc,
    Gpt_PrescaleType Gpt_ChannelPrescale,
    #if (GPT_DUAL_CLOCK_MODE == STD_ON)
    Gpt_PrescaleType Gpt_ChannelPrescale_Alternate,
    #endif
    boolean Gpt_FreezeEnable,
    boolean Gpt_EnableWakeup,
    Gpt_NotifyType Gpt_Notification,
    Gpt_WakeUpType Gpt_ChannelWakeUpInfo
} Gpt_LLD_ChannelConfigType;
```

Table 3-35. Structure Gpt_LLD_ChannelConfigType member description

Member	Description
Gpt_HW_Channel	GPT hw channel ID.
Gpt_HW_Module	GPT hw module used.
Gpt_ChannelMode	GPT channel mode.
Gpt_ChannelClkSrc	GPT clock source.
Gpt_ChannelPrescale	GPT ch prescaler value.
Gpt_ChannelPrescale_Alternate	ch 2nd prescaler val
Gpt_FreezeEnable	GPT ch freeze enable.
Gpt_EnableWakeup	GPT ch WakeUp enable.
Gpt_Notification	pointer to ISR function
Gpt_ChannelWakeUpInfo	EcuM wake up reference.

Note: Not all the parameters in this structure are available for all the possible hardware IPs

3.10.2.2 GptChannelId

Table 3-36. GptChannelId

Description	Channel Id of the GPT channel. This value will be assigned to the symbolic name derived of the GptChannelConfiguration container short name.
Class	Autosar Parameter
Range	0 -> 4294967295
Default	0
Source File	Gpt_Cfg.h
Source Representation	<code>#define GptChannelConfiguration_0 0</code>

3.10.2.3 GptSTMChannelPrescale

Table 3-37. GptSTMChannelPrescale

Description	Selects the clock divide value for the STM prescaler (1-256)
Class	Implementation specific Non-Autosar Parameter
Range	1 - 256
Default	1
Source File	Gpt_Cfg.c,Gpt_PBcfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(STM_CH_2), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_STM_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType)(0U), /* STM Clock divider */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.4 GptHwChannel

Table 3-38. GpHwChannel

Description	Selects the physical GPT Channel.
Class	Implementation specific Non-Autosar parameter
Range	EMIOS_0_CH_0 EMIOS_0_CH_1 EMIOS_0_CH_2 EMIOS_0_CH_3 EMIOS_0_CH_4 EMIOS_0_CH_5 EMIOS_0_CH_6 EMIOS_0_CH_7 EMIOS_0_CH_8 EMIOS_0_CH_16 EMIOS_0_CH_23 EMIOS_0_CH_24 EMIOS_1_CH_0 EMIOS_1_CH_8 EMIOS_1_CH_16

Table continues on the next page...

Table 3-38. GpHwChannel (continued)

	EMIOS_1_CH_23 EMIOS_1_CH_24 PIT_CH_0 PIT_CH_1 PIT_CH_2 PIT_CH_3 PIT_CH_4 PIT_CH_5 PIT_CH_6 PIT_CH_7 STM_CH_0 STM_CH_1 STM_CH_2 STM_CH_3 RTC_CH_0
Default	EMIOS_0_CH_0
Source File	Gpt_Cfg.h
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(STM_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_STM_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 1,/* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } } </pre>

3.10.2.5 GptChannelMode

Table 3-39. GptChannelMode

Description	Specifies the behaviour of the timer channel after the timeout has expired
Class	Autosar Parameter
Range	GPT_MODE_ONESHOT, GPT_MODE_CONTINUOUS
Default	GPT_MODE_ONESHOT
Source File	Gpt_Cfg.c,Gpt_PBCfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) </pre>

Table 3-39. GptChannelMode

	<pre> Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(STM_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_STM_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 1, /* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>
--	--

3.10.2.6 GptChannelTickFrequency

Table 3-40. GptChannelTickFrequency

Description	The parameter specifies the desired tick frequency of the timer channel in Hz. It is used for calculating the channel prescaling factor of the clock. It is active only for the STM, EMIOs and RTC channels. When the GPT channel is EMIOs, GptChannelTickFrequency is used to configure the eMIOS channel prescaler. In order to configure eMIOS/STM channel prescaler the following needs to be done in MCU and GPT modules: In MCU module: - from McuClockSettingConfig container, define EMIOs Settings for Global prescaler (which is fine so far) - then create from the McuClockReferencePoint container a config for the EMIOs clock reference (purpose here is just to compute the exact clock reference for EMIOs according to settings defined above) In GPT module: - create a new clock reference point that will refer back to the one just defined in MCU... - Now and only now you can define your tick frequency for a GPT channel based on eMIOS Note: Incase of RTC, the Prescaler is calculated based on the parameter'GptChannelClkSrc' and McuClockReferencePoint will not be used.
Class	Autosar Parameter
Range	The minimum value is limited by the prescaling factor. The maximum value is the module clock frequency.
Default	1.0
Source File	NA

3.10.2.7 GptChannelPrescale

Table 3-41. GptChannelPrescale

Description	GPT module specific channel prescaler factor. For this platform it is not available. Selects the clock divide value for the prescaler ONLY for EMIOs channel (value between 1- 4 only)
Class	Autosar Parameter

Table continues on the next page...

Table 3-41. GptChannelPrescale (continued)

Range	1-4
Default	1
Source File	Gpt_Cfg.c,Gpt_PBcfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(EMIOS_0_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_EMIOS_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType)EMIOS_GPT_LLD_CH_PRES_1,/* EMIOS Clock divider */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.8 GptFreezeEnable

Table 3-42. GptFreezeEnable

Description	Select to set Freeze enable for EMIOS, STM, PIT channels
Class	Implementation specific Non-Autosar Parameter
Range	True, False
Default	True
Source File	Gpt_Cfg.h
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(PIT_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_PIT_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 1,/* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.9 GptEnableWakeup

Table 3-43. GptEnableWakeup

Description	Enables wakeup capability of CPU for a channel when timeout period expires. This might be different to enabling the notification depending on hardware capabilities
Class	Autosar Parameter
Range	True, False
Default	False
Source File	Gpt_Cfg.c,Gpt_PBcfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(STM_CH_2), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_STM_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 1,/* Not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.10 GptNotification

Table 3-44. GptNotification

Description	Function pointer to callback function.Note:The field is editable only,If the switch GptEnableDisableNotificationApi is true.
Class	Autosar Parameter
Range	NA
Default	NULL_PTR
Source File	Gpt_Cfg.C,Gpt_PBcfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(STM_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_STM_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 1,/* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.11 Gpt_ChannelPrescale_Alternate

Table 3-45. Gpt_ChannelPrescale_Alternate

Description	This is a non AutoSAR feature implementation. GPT module specific prescaler factor per channel in dual mode. Selects the clock divide value for the prescaler.
Class	Implementation specific Non-Autosar Parameter
Range	1 - 4
Default	1
Source File	Gpt_Cfg.c,Gpt_PBCfg.c
Source Representation	<pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPC_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(EMIOS_0_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_EMIOS_MODULE), /* hardware module ID */ GPT_MODE_ONESHOT, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType)EMIOS_GPT_LLD_CH_PRES_1,/* EMIOS Clock divider */ #if GPT_DUAL_CLOCK_MODE==STD_ON (Gpt_PrescaleType)EMIOS_GPT_LLD_CH_PRES_1,/* EMIOS Clock Dual Mode divider */ #endif (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ NULL_PTR, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } }; </pre>

3.10.2.12 GptChannelClkSrcRef

Table 3-46. GptChannelClkSrcRef

Description	Reference to the GptClockReferencePoint, representing the module clock, from which the channel clock is derived.
Class	Autosar Parameter
Range	NA
Default	NA
Source File	NA

3.10.2.13 GptWakeupSourceRef

Table 3-47. GptWakeupSourceRef

Description	In case the wakeup-capability is true this value is transmitted to the Ecu State Manager. Implementation Type: reference to EcuM_WakeupSourceType
Class	Autosar Parameter
Range	NA
Default	NA
Source File	Gpt_Cfg.c,Gpt_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPB_1[GPT_CHANNEL_NUM_1]= { { (Gpt_ChannelType)(PIT_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_PIT_MODULE), /* hardware module ID */ GPT_MODE_CONTINUOUS, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 0,/* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ GptTestCallback03, /* Channel notification */ (Gpt_WakeUpType)(0) /* Wakeup information */ } } </pre>

3.10.2.14 GptDisableEcumWakeupSourceNotification

Table 3-48. GptDisableEcumWakeupSourceNotification

Description	In case the wakeup-capability is true, this parameter will Enable/Disable the Notification to Ecum.
Class	Implementation specific Non-Autosar Parameter
Range	NA
Default	NA
Source File	Gpt_Cfg.c,Gpt_PBcfg.c
Source Representation	The highlighted portion of the following structure: <pre> STATIC CONST(Gpt_LLD_ChannelConfigType, GPT_CONST) Gpt_InitChannelPB_1[GPT_CHANNEL_NUM_1]= { { </pre>

Table 3-48. GptDisableEcumWakeupSourceNotification

	<pre> (Gpt_ChannelType)(PITRTI_CH_0), /* GPT physical channel no. */ (Gpt_ChannelType)(GPT_PITRTI_MODULE), /* hardware module ID */ GPT_MODE_CONTINUOUS, /* Timer mode:continuous/one-shot */ (uint8)0, /* not used*/ (Gpt_PrescaleType) 0,/* not used */ (boolean)TRUE, /* Freeze Enable */ (boolean)FALSE, /* Wakeup capability */ GptTestCallback03, /* Channel notification */ (Gpt_WakeUpType)(0), /* Wakeup information */ (boolean)FALSE /* Enable/Disable Ecum Notification */ } </pre>
--	--

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13GPTASR4.0R1.0.1
Revision 1.2